

Homework 7

You may submit your solution to this homework assignment in the text window in iCollege, as a `.txt` file, or in some other commonly used format, *e.g.*, `.pdf`, `.docx`, *etc.*

In this homework assignment, we use regular expressions to study the full text of the book *Harry Potter and the Sorcerer's Stone*, which can be found at

<https://raw.githubusercontent.com/murraypatterson/fixed-example/main/HarryPotter.txt>

1. In the text:

- (a) How many lines contain the literal string `Harry` ?
- (b) How many instances of the literal string `Harry` are there?

Note that the count for (a) should be less than (or equal to) the count for (b)—this is one way to check for correctness.

2. In the text, how many times does the *word* “had” appear? Unlike the first exercise, which was asking for instances of the literal string `Harry`, in this exercise, the goal is to find instances of the *word* “had”, even if it begins a sentence or quote (*e.g.*, strings `Had` and `"Had"`), is at the end of a sentence or clause (*e.g.*, strings `had.` and `had,`), *etc.* Moreover, strings such as `hadn't`, `shade` or `shadows` do *not* count as instances of the word “had”.

See the following examples of tricky sentences which appear in the text, and the number of times the *word* “had” appears in each such sentence:

- i. Had that been what the thieves were looking for? (1 time)
- ii. And so he had. (1 time)
- iii. Hedwig hadn't brought Harry anything so far. (0 times)
- iv. Then a small voice came out of the shadows. (0 times)
- v. Mrs. Dursley had had a nice, normal day. (2 times)

Note: that you may do this in several steps if you wish, *e.g.*, perform a regex whose result can go to some intermediate file, and then perform another regex on this intermediate file, *etc.*

3. Remember the regular language of exercise 2. of Homework 6—the language with an odd number of `a`'s? Now we want to generalize it to larger alphabets. In the text, how many *words* have an odd number of `A`'s/`a`'s? Here, *word* is defined as a contiguous set of printing characters including apostrophes `'` and hyphens `-`, where contiguity is broken by any whitespace character including space, tab, *etc.*

Note: watch out for contractions such as `can't` or `shouldn'ta` (Hagrid has somewhat of a thick accent) as well as compound words such as `last-minute` or `best-played`. There is even an instance of `what's-her-name`, which has both `'` and `-` (but would be a word with an even number of `A`'s/`a`'s according to the definition above).

Also: note, again, that you may do this in several steps if you wish, using any programming language and its features (not just regexes), *e.g.*, Python's `split()` function to split the text into some intermediate file, perform a regex on this file to produce a second intermediate file, *etc.* See the working examples in the slides “regexes-in-practice.pdf” for some inspiration.

4. Remember the regular languages of exercises 1. and 2. of Homework 6? Show that both of these languages are context-free languages by giving a corresponding context-free grammar for each. That is, show that:
- (a) The language composed of all strings over the alphabet $\{a, b\}$ of the form: zero or more a 's followed by zero or more b 's, *i.e.*, $\{\varepsilon, a, b, aa, ab, abb, aaa, aab, abb, bbb, \dots\}$, where ε is the empty string, is context-free.
 - (b) The language composed of all strings over the alphabet $\{a, b\}$ that contain an odd number of a 's, is context-free.

Note that stating just the production rules is sufficient to describe the context-free grammar.

5. Consider the language composed of all strings over the alphabet $\{a, b\}$ that contain twice as many b 's as a 's, *i.e.*, $\{\varepsilon, abb, bab, bba, aabbbb, ababbb, abbabb, abbbab, \dots\}$ (note that the empty string ε is there for completeness, *i.e.*, twice as many as zero is zero). Show that this language is context-free by giving a corresponding context-free grammar.